

Valós számok ábrázolása számítógépen

Gépi számábrázolás, IEEE 754, hibaszámítás

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok

- Gyakorlati és tudományos számításokban szinte mindig **valós számokra** van szükségünk.
- A számítógépen viszont csak egy **véges halmaz** elemei közül választhatunk.
- Ráadásul az ábrázolandó mennyiségek több **nagyságrenddel** eltérhetnek egymástól:

$$c \approx 3 \cdot 10^8 \text{ m/s}, \quad r_{\text{atom}} \approx 10^{-10} \text{ m}.$$

- A véges ábrázolás következményei néha meglepőek. Nézzünk néhány példát!

Mennyi $\sin(\pi)$ értéke?

Gép (double) szerint

$$\sin(\pi) = 1.224646799147353e-16$$

Mennyi $\sin(\pi)$ értéke?

Gép (double) szerint

```
sin(pi) = 1.224646799147353e-16
```

Pedig $\sin(\pi) = 0$ pontosan.

- Vajon a $\sin$ függvény implementációja rossz?
- Vagy valami más okozza a hibát?

2. furcsa jelenség: harmonikus sor

Mennyi $\sum_{k=1}^{+\infty} \frac{1}{k}$ értéke? És mennyi az n -edik részletösszeg, $S_n = \sum_{k=1}^n \frac{1}{k}$ valamely nagy n -re?

Összegezzük oda vagy vissza... $n = 10^7$, egyszeres pontosságú (float) aritmetikával:

összegzés iránya	eredmény
$k = 1, 2, \dots, n$ (növekvő)	15.403683
$k = n, n - 1, \dots, 1$ (csökkenő)	16.686031
pontos érték	16.695311...

2. furcsa jelenség: harmonikus sor

Mennyi $\sum_{k=1}^{+\infty} \frac{1}{k}$ értéke? És mennyi az n -edik részletösszeg, $S_n = \sum_{k=1}^n \frac{1}{k}$ valamely nagy n -re?

Összegezzük oda vagy vissza... $n = 10^7$, egyszeres pontosságú (float) aritmetikával:

összegzés iránya	eredmény
$k = 1, 2, \dots, n$ (növekvő)	15.403683
$k = n, n - 1, \dots, 1$ (csökkenő)	16.686031
pontos érték	16.695311...

Az összeadás **nem asszociatív** a gépen: a sorrend számít!

3. furcsa jelenség: egy rekurzió

Mennyi a

$$T_n := \int_0^1 f_n = \int_0^1 \frac{x^n}{x+10} dx$$

határozott integrál értéke? (Az integrandus pozitív, tehát $T_n > 0$, sőt T_n monoton csökken.)

Parciális átalakítással:

$$T_n + 10 T_{n-1} = \int_0^1 \frac{x^n + 10x^{n-1}}{x+10} dx = \int_0^1 x^{n-1} dx = \frac{1}{n},$$

azaz

$$\boxed{T_n = \frac{1}{n} - 10 \cdot T_{n-1}}, \quad T_0 = \ln 1.1.$$

Számoljuk ki!

3. furcsa jelenség: az eredmény

n	T_n (gépi)	n	T_n (gépi)
0	0.095310	15	-0.069122
5	0.015353	16	0.753721
10	0.008329	17	-7.478389
13	0.005785	18	74.839443
14	0.013579	19	-748.341

A sorozatnak monoton csökkenőnek és pozitívnak kellene lennie — ehelyett előjelet vált és **divergens**.

- T_{18} pontos értéke kb. 0.005.
- Az egyetlen hibaforrás T_0 kerekítése volt. . .

4. furcsa jelenség: két közeli szám különbsége

Mennyi $\sqrt{2014} - \sqrt{2013}$ értéke? Alakítsuk át:

$$(\sqrt{2014} - \sqrt{2013}) \cdot \frac{\sqrt{2014} + \sqrt{2013}}{\sqrt{2014} + \sqrt{2013}} = \frac{2014 - 2013}{\sqrt{2014} + \sqrt{2013}} = \frac{1}{\sqrt{2014} + \sqrt{2013}}.$$

Matematikailag a két alak azonos. A gépen viszont:

képlet	double eredmény
$\sqrt{2014} - \sqrt{2013}$	0.011142796284637768
$1/(\sqrt{2014} + \sqrt{2013})$	0.011142796284640395

Az utolsó 3 jegy eltér: ez a **kivonási jegyvesztés** (*catastrophic cancellation*).

5. furcsa jelenség: a „pénzprobléma”

Klasszikus

`0.1 + 0.2 == 0.3` $\implies$ `false`

`0.1 + 0.2 = 0.30000000000000004`

- A 0.1 kettes számrendszerben **végtelen szakaszos tört**:

$$0.1_{10} = 0.0\overline{0011}_2.$$

- Ahogy $1/3$ -ot sem tudjuk véges tizedes törttel leírni, úgy $1/10$ -et sem véges sok számjeggyel a kettes számrendszerben.
- Ezért **pénzügyi elszámolásban** soha ne használjunk bináris lebegőpontos típust: fixpontos egész (fillér/cent) vagy decimális típus (`decimal`, `BigDecimal`) kell.

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje**
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok

- **Fixpontos** ábrázolásnál a (bináris) pont helye rögzített. Pl. 8 biten $4 + 4$ bites felosztással:

$$\underbrace{0011}_{\text{egész}} . \underbrace{1000}_{\text{tört}} = 3 + \frac{1}{2} = 3.5.$$

- A lépköz állandó (itt $1/16$), a tartomány szűk ($0 \dots 15$).
- **A tartomány és a pontosság egymás rovására megy:** ha nagy számokat akarunk ábrázolni, elveszítjük a törtrész pontosságát, és fordítva.
- Beágyazott rendszerekben, hanganyagokkal kapcsolatos alkalmazásokban ma is használják, általános célra viszont alkalmatlan.

A lebegőpontos ábrázolás alapelve

Az ötlet a **normálalakból** (tudományos jelölés) származik:

$$324 \rightsquigarrow +0.324 \cdot 10^3, \quad 0.00047 \rightsquigarrow +0.47 \cdot 10^{-3}.$$

Kettes számrendszerben ugyanígy:

$$101000100_2 \rightsquigarrow +0.101000100 \cdot 2^9.$$

- A bináris pont „**lebeg**”: helyét az exponens (kitevő) adja meg.
- Így ugyanazon a típuson belül ábrázolható a fénysebesség és egy atom átmérője is.
- Ára: az ábrázolható számok **nem egyenletesen** helyezkednek el a számegyenesen.

Általános alak:

$$\pm 0. \underbrace{1 _ \dots _}_{t \text{ jegy}} \cdot 2^k, \quad (k^- \leq k \leq k^+).$$

Definíció: normalizált lebegőpontos szám

Legyen

$$m = \sum_{i=1}^t m_i \cdot 2^{-i}, \quad t \in \mathbb{N}, \quad m_1 = 1, \quad m_i \in \{0, 1\}.$$

Ekkor az $a = \pm m \cdot 2^k$ ($k \in \mathbb{Z}$) alakú számot **normalizált lebegőpontos számnak** nevezzük.

- m : a szám *mantisszája*, hossza t
- k : a szám *karakterisztikája* (kitevője), $k^- \leq k \leq k^+$

Jelölés: $a = \pm [m_1 m_2 \dots m_t \mid k] = \pm 0.m_1 m_2 \dots m_t \cdot 2^k$.

Jelölés: $\mathbb{M} = \mathbb{M}(t, k^-, k^+)$ a gépi számok halmaza adott $k^-, k^+ \in \mathbb{Z}$ és $t \in \mathbb{N}$ esetén. (Általában $k^- < 0$ és $k^+ > 0$.)

Definíció: gépi számok halmaza

$$\mathbb{M}(t, k^-, k^+) = \left\{ a = \pm 2^k \cdot \sum_{i=1}^t m_i \cdot 2^{-i} : \begin{array}{l} k^- \leq k \leq k^+, \\ m_i \in \{0, 1\}, m_1 = 1 \end{array} \right\}$$

A gyakorlatban még hozzávesszük: 0 , $+\infty$, $-\infty$, NaN, valamint a *denormalizált* számokat.

Példa

$$\mathbb{M}(3, -1, 2) = 0.1_ _ \cdot 2^k, \quad (-1 \leq k \leq 2)$$

- Elemei $k = 0$ esetén: 0.100, 0.101, 0.110, 0.111, azaz $\frac{1}{2}, \frac{5}{8}, \frac{6}{8}, \frac{7}{8}$.
- $k = -1$ esetén ezek fele, $k = 1$ esetén kétszerese, $k = 2$ esetén négyszerese. (Továbbá negatív előjellel...)

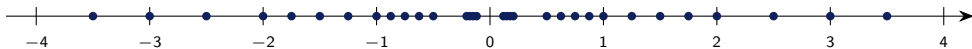
$$\varepsilon_0 = [100|-1] = 0.100 \cdot 2^{-1} = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4} = 0.25$$

$$\varepsilon_1 = [101|1] - 1 = 0.101 \cdot 2^1 - 1 = 0.01_2 = \frac{1}{4} = 0.25$$

$$M_\infty = [111|2] = 0.111 \cdot 2^2 = \frac{7}{8} \cdot 4 = \frac{7}{2} = 3.5, \quad |\mathbb{M}| = 2 \cdot 2^2 \cdot 4 = 32$$

A gépi számok a számegyenesen

$$\mathbb{M}(3, -1, 2), \quad |\mathbb{M}| = 32$$



a 0 nem eleme $\mathbb{M}$ -nek, külön vesszük hozzá

- A gépi számok **sűrűbben** helyezkednek el a 0 közelében, és egyre ritkábban távolodva.
- Két szomszédos kitevő között mindig ugyanannyi (2^{t-1}) gépi szám van.
- A 0 körül nagy „lyuk” van: $(-\varepsilon_0, \varepsilon_0)$ üres (ezt enyhítik a denormalizált számok).

Gépi számok tulajdonságai, nevezetes értékei

① $\frac{1}{2} \leq m < 1$ (a normalizáltság miatt)

② $\mathbb{M}$ szimmetrikus a 0-ra.

③ $\mathbb{M}$ legkisebb pozitív eleme:

$$\varepsilon_0 = [100 \dots 0 \mid k^-] = \frac{1}{2} \cdot 2^{k^-} = 2^{k^- - 1}$$

④ $\mathbb{M}$ -ben az 1 után következő gépi szám és 1 különbsége (*gépi epszilon*):

$$\varepsilon_1 = [100 \dots 01 \mid 1] - [100 \dots 00 \mid 1] = 2^{-t} \cdot 2^1 = 2^{1-t}$$

⑤ $\mathbb{M}$ legnagyobb eleme:

$$M_\infty = [111 \dots 11 \mid k^+] = (1 - 2^{-t}) \cdot 2^{k^+}$$

⑥ $\mathbb{M}$ elemeinek száma (számossága):

$$|\mathbb{M}| = 2 \cdot 2^{t-1} \cdot (k^+ - k^- + 1)$$

Valós számok ábrázolása: az input függvény

Hogyan feleltetünk meg egy $\mathbb{R}$ -beli számnak egy gépi számot?

Definíció: input függvény

Az $\text{fl}: \mathbb{R} \rightarrow \mathbb{M}$ függvényt **input függvénynek** nevezzük, ha

$$\text{fl}(x) = \begin{cases} 0 & \text{ha } |x| < \varepsilon_0, \\ +\infty & \text{ha } x > M_\infty, \\ -\infty & \text{ha } x < -M_\infty, \\ \tilde{x} & \text{ha } \varepsilon_0 \leq |x| \leq M_\infty, \end{cases}$$

ahol $\tilde{x}$ az x -hez legközelebbi gépi szám (a kerekítés szabályai szerint).

Az első eset az **alulcsordulás** (underflow), a második-harmadik a **túlcsordulás** (overflow).

Tehát már az is hibát okoz, hogy a valós számokat számítógépre visszük. . . de mekkorát?

Tétel: input hiba

Minden $x \in \mathbb{R}$, $\varepsilon_0 \leq |x| \leq M_\infty$ esetén

$$|x - \text{fl}(x)| \leq |x| \cdot 2^{-t} = \frac{1}{2} \cdot |x| \cdot \varepsilon_1,$$

illetve

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{1}{2} \cdot \varepsilon_1.$$

- A **relatív** hiba korlátja tehát független x -től!
- A hiba lényegében ε_1 -től, azaz t -től függ.
- Az $u := \frac{1}{2}\varepsilon_1 = 2^{-t}$ mennyiség neve: *egységnyi kerekítési hiba* (unit roundoff).

Az alapl műveletek eredményét is kerekíteni kell. Az IEEE 754 által megkövetelt („helyesen kerekített”) aritmetikában $\odot \in \{+, -, \cdot, /\}$ esetén

$$\text{fl}(a \odot b) = (a \odot b)(1 + \delta), \quad |\delta| \leq u = \frac{1}{2}\varepsilon_1.$$

Vagyis **egyetlen** művelet relatív hibája minimális — a baj akkor kezdődik, amikor sok műveletet fűzünk össze.

Következmény

Gépi számokkal végzett műveletek során...

- sérülhet a pozitivitás: $\exists a, b \in \mathbb{M}, a, b > 0 : a + b = a$
- sérülhet az asszociativitás: $\exists a, b, c \in \mathbb{M} : (a + b) + c \neq a + (b + c)$
- sérülhet a disztributivitás is.

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány**
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok

- A '70-es évekig minden gyártó másképp ábrázolta a valós számokat: eltérő kerekítés, eltérő túl- és alulcsordulás-kezelés.
- Ugyanaz a program ugyanazon a bemeneten **más eredményt** adott két különböző gépen $\Rightarrow$ komoly reprodukálhatósági probléma.
- **1985**: megszületik az *IEEE 754 — Standard for Floating-Point Arithmetic* (W. Kahan és mtsai; Kahan 1989-ben Turing-díjat kapott).
- Fontosabb revíziók: **IEEE 754-2008** (binary16, binary128, decimális formátumok, FMA), **IEEE 754-2019**.
- Ma gyakorlatilag minden CPU/GPU ezt a szabványt követi.

Egy IEEE 754 szám három részből áll: $x = (-1)^S \cdot 1.M \cdot 2^{E-\text{bias}}$



- **Előjel** (S): 0 = pozitív, 1 = negatív.
- **Karakterisztika** (E): *eltolt* (biased) formában tárolva, hogy mindig nemnegatív legyen $\Rightarrow$ a lebegőpontos számok egészként is összehasonlíthatók.
- **Mantissza** (M): a szám értékes jegyei. Normalizált esetben a vezető 1-es *implicit*, nem tároljuk $\Rightarrow$ egy bit nyereség.

Formátum	bitek	S	E	M	bias	pontosság
binary16 (half)	16	1	5	10	15	~ 3–4 tizedesjegy
binary32 (float)	32	1	8	23	127	~ 6–9 tizedesjegy
binary64 (double)	64	1	11	52	1023	~ 15–17 tizedesjegy
binary128 (quad)	128	1	15	112	16383	~ 34 tizedesjegy

	binary32	binary64
legkisebb pozitív normalizált ε_0	$1.18 \cdot 10^{-38}$	$2.22 \cdot 10^{-308}$
legnagyobb M_∞	$3.40 \cdot 10^{38}$	$1.80 \cdot 10^{308}$
gépi epszilon $\varepsilon_1 = 2^{1-t}$	$2^{-23} \approx 1.19 \cdot 10^{-7}$	$2^{-52} \approx 2.22 \cdot 10^{-16}$

Az x87 80 bites *extended* formátum (1 + 15 + 64, explicit vezető bit) a C/C++ long double.
A binary16 a gépi tanulás/GPU-k miatt lett népszerű.

Az előadás modelljében $m \in [\frac{1}{2}, 1)$, az IEEE-ben $1.M \in [1, 2)$ — a kettő között csak egy 2-es faktor a különbség:

$$1.f \cdot 2^E = \underbrace{\frac{1.f}{2}}_{\in [1/2, 1)} \cdot 2^{E+1}.$$

	durva megfeleltetés	pontosan (implicit bittel)
float	$\mathbb{M}(23, -128, 127)$	$\mathbb{M}(24, -125, 128)$
double	$\mathbb{M}(52, -1024, 1023)$	$\mathbb{M}(53, -1021, 1024)$

Ellenőrzés double-re:

$$\varepsilon_1 = 2^{1-53} = 2^{-52} \approx 2.22 \cdot 10^{-16}, \quad \varepsilon_0 = 2^{-1021-1} = 2^{-1022} \approx 2.23 \cdot 10^{-308},$$

$$M_\infty = (1 - 2^{-53}) \cdot 2^{1024} \approx 1.798 \cdot 10^{308}. \quad \checkmark$$

Denormalizált számok és különleges értékek

- **Denormalizált (szubnormális) számok:** ha E minden bitje 0, akkor az implicit vezető bit is 0 (mantissza: $0.M$). Ez teszi lehetővé a *fokozatos alulcsordulást* (gradual underflow): a 0 körüli „lyuk” kitöltését. Ára: sok hardveren lassabb feldolgozás.
- **± 0 :** minden E - és M -bit nulla; az előjelbit dönt. Matematikailag $+0 = -0$, de pl. $1/(+0) = +\infty$, $1/(-0) = -\infty$.
- **$\pm \infty$:** minden E -bit 1, minden M -bit 0. Túlcsordulás vagy nemnulla / nulla eredménye.
- **NaN** (Not a Number): minden E -bit 1, $M \neq 0$. Keletkezés: $0/0$, $\infty - \infty$, $\sqrt{-1}$, $\log(-1)$. . .
 - *qNaN* (quiet): csendben terjed a számításban;
 - *sNaN* (signaling): kivételt vált ki.

Fontos: **NaN** $\neq$ **NaN**, minden összehasonlítás hamis rá.

Példa: 12.625 ábrázolása binary32-ben

1. Kettes számrendszer. Egész rész: $12_{10} = 1100_2$. Törtrész ismételt szorzással:

$$0.625 \cdot 2 = \mathbf{1.25}, \quad 0.25 \cdot 2 = \mathbf{0.5}, \quad 0.5 \cdot 2 = \mathbf{1.0} \Rightarrow 0.101_2.$$

Tehát $12.625_{10} = 1100.101_2$.

2. Normalizálás. $1100.101_2 = 1.100101_2 \cdot 2^3$.

3. Mezők.

- $S = 0$ (pozitív)
- $E = 3 + 127 = 130 = 10000010_2$
- $M = 100101$, nullákkal feltöltve 23 bitre: 10010100000000000000000

0 10000010 10010100000000000000000

Ez a szám **pontosan** ábrázolható, mert a törtrész kettedes törtje véges.

Példa: 0.1 ábrázolása — ami nem megy pontosan

$$0.1 \cdot 2 = \mathbf{0.2} \quad 0.2 \cdot 2 = \mathbf{0.4} \quad 0.4 \cdot 2 = \mathbf{0.8}$$

$$0.8 \cdot 2 = \mathbf{1.6} \quad 0.6 \cdot 2 = \mathbf{1.2} \quad 0.2 \cdot 2 = \mathbf{0.4} \quad (\text{ismétlődik})$$

$$0.1_{10} = 0.0001100110011 \dots_2 = 1.100110011 \dots_2 \cdot 2^{-4}$$

- $S = 0$, $E = -4 + 127 = 123 = 01111011_2$,
- $M = 10011001100110011001100$ (23 bit, **kerekítve**).

A tárolt érték valójában $0.100000001490116119384765625$ (float esetén).

Tanulság

A legtöbb tizedes tört **nem** ábrázolható pontosan binárisan — ahogy $1/3$ sem írható le véges tizedes törttel.

Az IEEE 754 négy (2008-tól öt) kerekítési módot definiál:

- 1 legközelebbire, döntetlennél párosra (*round half to even*) — ez az **alapértelmezett**;
- 2 legközelebbire, döntetlennél nullától el;
- 3 $+\infty$ felé (felfelé);
- 4 $-\infty$ felé (lefelé);
- 5 0 felé (csonkolás).

Miért „párosra”?

Ha mindig felfelé kerekítenénk döntetlennél, a hibák **rendszeresen egy irányba** halmozódna. A párosra kerekítés átlagosan kiegyenlíti a fel- és lefelé kerekítéseket, így hosszú számítási láncokban kisebb a szisztematikus torzítás.

Összeadás/kivonás:

- 1 exponensek kiegyenlítése (a kisebbik mantisszáját jobbra toljuk),
- 2 mantisszák összeadása/kivonása,
- 3 normalizálás,
- 4 kerekítés.

Pl. $(1.23 \cdot 10^3) + (4.56 \cdot 10^1) = (1.23 + 0.0456) \cdot 10^3 = 1.2756 \cdot 10^3$.

Szorzás/osztás: exponensek összeadása/kivonása (a bias korrekciójával), mantisszák szorzása/osztása, előjel: XOR, majd normalizálás és kerekítés.

- A szorzás/osztás **jól viselkedik**: a relatív hibák egyszerűen összeadódnak.
- Az **összeadás/kivonás** a veszélyes: ott léphet fel jegyvesztés, illetve „elnyelődhet” a kisebbik tag.

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei**
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok

Definíció: hibák jellemzése

Legyen A egy pontos érték, a pedig egy közelítő értéke. Ekkor:

$\Delta a := A - a$ a közelítő érték (pontos) hibája,

$|\Delta a| := |A - a|$ a közelítő érték abszolút hibája,

$\Delta_a \geq |\Delta a|$ az a egy abszolút hibakorlátja,

$\delta a := \frac{\Delta a}{A} \approx \frac{\Delta a}{a}$ az a relatív hibája,

$\delta_a \geq |\delta a|$ az a egy relatív hibakorlátja.

Példa

Vizsgáljuk meg a 3.14 számot mint a π egy közelítő értékét!

$$A = \pi = 3.14159265 \dots, \quad a = 3.14.$$

$$\Delta a = \pi - 3.14 = 0.0015926 \dots > 0 \quad (\text{alulról közelítünk}),$$

$$|\Delta a| = 0.0015926 \dots,$$

$$\Delta_a = 0.0016 \quad (\text{egy lehetséges abszolút hibakorlát}),$$

$$\delta a = \frac{0.0015926}{3.14159} = 5.069 \cdot 10^{-4},$$

$$\delta_a = 5.1 \cdot 10^{-4} \approx 0.05 \, \%.$$

- Az abszolút hiba **mértékegységes**, a relatív hiba dimenziótlan.
- Nagyságrendekben eltérő adatoknál a **relatív** hiba a beszédesebb: 1 méter hiba a Föld–Nap távolságban jelentéktelen, egy asztal hosszában végzetes.

Tétel: az alpműveletek hibakorlátai

$$\Delta_{a \pm b} = \Delta_a + \Delta_b \qquad \delta_{a \pm b} = \frac{|a| \cdot \delta_a + |b| \cdot \delta_b}{|a \pm b|}$$

$$\Delta_{a \cdot b} = |b| \cdot \Delta_a + |a| \cdot \Delta_b \qquad \delta_{a \cdot b} = \delta_a + \delta_b$$

$$\Delta_{a/b} = \frac{\Delta_{a \cdot b}}{b^2} \qquad \delta_{a/b} = \delta_a + \delta_b$$

- Szorzásnál és osztásnál a **relatív** hibák adódnak össze — ez „jóindulatú” viselkedés.
- Összeadásnál/kivonásnál az **abszolút** hibák adódnak össze, de a relatív hiba nevezőjében $|a \pm b|$ áll: ha ez közel van a 0-hoz, a relatív hiba **megnővekedhet**.

Biz.: csak a kivonásra és a szorzásra.

$$\Delta(a - b) = (A - B) - (a - b) = (A - a) - (B - b) = \Delta a - \Delta b$$

$$|\Delta(a - b)| \leq |\Delta a| + |\Delta b| \leq \Delta_a + \Delta_b$$

$$\frac{\Delta(a - b)}{a - b} = \frac{\Delta a - \Delta b}{a - b} = \frac{a \cdot \delta a - b \cdot \delta b}{a - b}$$

$$\frac{|\Delta(a - b)|}{|a - b|} = \left| \frac{a \cdot \delta a - b \cdot \delta b}{a - b} \right| \leq \frac{|a| \cdot |\delta a| + |b| \cdot |\delta b|}{|a - b|}$$



$$\begin{aligned}
 \Delta(a \cdot b) &= A \cdot B - a \cdot b = A \cdot B - A \cdot b + A \cdot b - a \cdot b = \\
 &= A(B - b) + b(A - a) = A \cdot \Delta b + b \cdot \Delta a = \\
 &= (a + \Delta a) \cdot \Delta b + b \cdot \Delta a \approx a \cdot \Delta b + b \cdot \Delta a \\
 &\quad (\Delta a \cdot \Delta b \text{ elhanyagolható})
 \end{aligned}$$

$$|\Delta(a \cdot b)| \leq |a| \cdot |\Delta b| + |b| \cdot |\Delta a| \leq |a| \cdot \Delta_b + |b| \cdot \Delta_a$$

$$\delta(a \cdot b) = \frac{\Delta(a \cdot b)}{a \cdot b} \approx \frac{a \cdot \Delta b + b \cdot \Delta a}{a \cdot b} = \frac{\Delta b}{b} + \frac{\Delta a}{a} = \delta b + \delta a$$

$$|\delta(a \cdot b)| \leq |\delta a| + |\delta b| \leq \delta_a + \delta_b$$



Tétel: függvényérték hibája

Ha $f \in D(k_{\Delta_a}(a))$, akkor $\Delta_{f(a)} = M_1 \cdot \Delta_a$, ahol

$$M_1 = \sup \left\{ |f'(\xi)| : \xi \in k_{\Delta_a}(a) \right\}.$$

Biz.: A Lagrange-féle középértéktétel felhasználásával:

$$|f(A) - f(a)| = |f'(\xi)| \cdot |A - a| \leq M_1 \cdot \Delta_a.$$



Több változóra (teljes differenciál alapján):

$$\Delta_{f(a_1, \dots, a_n)} = \sum_{i=1}^n \sup \left| \frac{\partial f}{\partial x_i} \right| \cdot \Delta_{a_i}.$$

A relatív hibákra átfogalmazva: a hiba **felnagyítási tényezője**

$$\kappa(x) = \left| \frac{x \cdot f'(x)}{f(x)} \right| \quad \implies \quad \delta_{f(x)} \approx \kappa(x) \cdot \delta_x.$$

- κ kicsi $\implies$ a feladat **jól kondicionált**;
- κ nagy $\implies$ **rosszul kondicionált**: a bemenet apró bizonytalansága is nagy kimeneti hibát okoz.

Példa

$f(x) = \sin x$ esetén $\kappa(x) = \left| \frac{x \cos x}{\sin x} \right|$. Az $x = \pi$ pont közelében $\sin x \rightarrow 0$, tehát $\kappa \rightarrow \infty$: a $\sin$ kiértékelése π körül **rosszul kondicionált** feladat.

Fontos elkülöníteni: a *kondicionáltság* a feladat, a *stabilitás* az algoritmus tulajdonsága.

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre**
- 6 Gyakorlati tanácsok

1. $\sin(\pi)$ — a bemenet hibája

- A π irracionális, tehát **nem gépi szám**: $\text{fl}(\pi) \neq \pi$, és

$$|\pi - \text{fl}(\pi)| \lesssim \frac{1}{2}\varepsilon_1 \cdot \pi \approx 1.22 \cdot 10^{-16}.$$

- A gép nem $\sin(\pi)$ -t, hanem $\sin(\text{fl}(\pi))$ -t számolja ki. Mivel $\sin$ a π körül lineáris (meredeksége -1):

$$\sin(\text{fl}(\pi)) \approx \pi - \text{fl}(\pi) \approx 1.2246 \cdot 10^{-16}.$$

- Vagyis a kapott érték **teljesen helyes**: a $\sin$ implementáció pontosan azt adta, amit kellett — csak nem abban a pontban, amit gondoltunk.
- Az **abszolút** hiba parányi, a **relatív** hiba viszont 100% (hiszen a pontos érték 0).

2. Harmonikus sor — elnyelődés és összegzési sorrend

- Ha S már nagy és $1/k$ már kicsi, akkor $\text{fl}(S + 1/k) = S$: a tag **elnyelődik**. Ez pontosan a $\exists a, b > 0 : a + b = a$ jelenség.
- float esetén ez $S \approx 15$ -nél, $k \approx 2 \cdot 10^6$ körül következik be — onnantól a részletösszeg **nem változik többé**, hiába összegzünk tovább.
- Ha **hátról előre** (a kis tagoktól a nagyok felé) összegzünk, a kis tagok először egymással adódnak össze, így nem vesznek el $\Rightarrow$ pontosabb eredmény.
- Még jobb: **Kahan-féle kompenzált összegzés**, amely külön változóban tárolja és visszavezeti az elveszett bitet.

Ökölszabály

Sok tag összegzésénél a **nagyságrend szerint növekvő** sorrend (vagy kompenzált összegzés) a jó.

3. A rekurzió — instabil algoritmus

$$T_n = \frac{1}{n} - 10 \cdot T_{n-1}$$

Ha T_0 hibája Δ_0 , akkor T_n hibája:

$$\Delta_n = -10 \cdot \Delta_{n-1} \implies |\Delta_n| = 10^n \cdot |\Delta_0|.$$

- $|\Delta_0| \approx 10^{-17}$ esetén már $n = 16$ körül $|\Delta_n| \approx 10^{-1}$: nagyobb, mint maga a keresett érték!
- A képlet matematikailag helyes, az **algoritmus instabil**.

Megoldás: fordítsuk meg az irányt

$$T_{n-1} = \frac{1}{10} \left(\frac{1}{n} - T_n \right) \implies |\Delta_{n-1}| = \frac{|\Delta_n|}{10}$$

Induljunk pl. $T_{40} \approx 0$ -ból (durva becslés!), és számoljunk visszafelé: a hiba **lépésenként tizedelődik**, T_{20} -ra már 10^{-20} -nál kisebb.

4. $\sqrt{2014} - \sqrt{2013}$ — kivonási jegyvesztés

A relatív hibakorlát képlete:

$$\delta_{a-b} = \frac{|a|\delta_a + |b|\delta_b}{|a-b|}.$$

Itt $a \approx b \approx 44.9$, de $|a-b| \approx 0.0111$, tehát a nevező **négy nagyságrenddel** kisebb $\Rightarrow$ a relatív hiba kb. 8000-szeresére nő.

- Szemléletesen: a két szám vezető értékes jegyei **kioltják** egymást, és az eredmény a mantissa végén lévő, már hibával terhelt bitekből épül fel.
- **Megoldás:** algebrai átalakítás, amely elkerüli a kivonást:

$$\sqrt{2014} - \sqrt{2013} = \frac{1}{\sqrt{2014} + \sqrt{2013}}.$$

- Hasonló klasszikus eset a másodfokú megoldóképlet $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, ha $b^2 \gg 4ac$: az egyik gyököt a $x_1 x_2 = c/a$ Viète-formulából érdemes számolni.

- 1 **Modellhiba** — a valóság matematikai leírásának hibája.
- 2 **Öröklött hiba** — a bemenő adatok mérési vagy ábrázolási (input) hibája.
- 3 **Képlethiba (csonkítási hiba)** — végtelen folyamat (sor, iteráció, integrál) véges lépéssel való helyettesítése.
- 4 **Kerekítési hiba** — a véges számábrázolás és a gépi műveletek hibája.

Két kulcsfogalom

Kondicionáltság a *feladat* érzékenysége a bemenet megváltoztatására — ezen az algoritmus nem segít.

Stabilitás az *algoritmus* tulajdonsága: felnagyítja-e a menet közben keletkező kerekítési hibákat. Ezen lehet javítani!

- 1 „Furcsa” jelenségek
- 2 Gépi számok: a lebegőpontos számok egy modellje
- 3 Az IEEE 754 szabvány
- 4 A hibaszámítás elemei
- 5 Magyarázat a „furcsa” jelenségekre
- 6 Gyakorlati tanácsok**

Soha ne hasonlítsunk lebegőpontos számokat egyenlőségre!

```
# ROSSZ
if x == 0.3: ...

# JOBB: kombinált abszolút es relatív türeshatar
import math
if math.isclose(x, 0.3, rel_tol=1e-9, abs_tol=1e-12): ...

# vagy kezzel:
if abs(a - b) <= eps * max(1.0, abs(a), abs(b)): ...
```

- Csak abszolút tűréshatár: nagy számoknál használhatatlan.
- Csak relatív tűréshatár: a 0 közelében használhatatlan.
- Ciklusváltozó **soha** ne legyen lebegőpontos (for x = 0 : 0.1 : 1 → egész indexelés!).

```
def kahan_sum(xs):  
    s = 0.0          # a futo osszeg  
    c = 0.0          # az eddig "elveszett" resz  
    for x in xs:  
        y = x - c      # a kompenzalt tag  
        t = s + y      # itt vesznek el bitek  
        c = (t - s) - y # ...es itt nyerjuk vissza oket  
        s = t  
    return s
```

- Költsége kb. $4 \times$ művelet, cserébe a hibakorlát $\mathcal{O}(n u)$ helyett gyakorlatilag $\mathcal{O}(u)$.
- A numpy, Julia, Python (`math.fsum`) beépített összegzései hasonló technikát használnak.

```
% --- gepi jellemzok ---  
eps                % 2.2204e-16 (a gepi epszilon)  
realmin, realmax   % 2.2251e-308, 1.7977e+308  
sin(pi)           % 1.2246e-16  
  
% --- 3. furcsa jelenseg: instabil rekurzio ---  
T = log(1.1);  
for n = 1:20  
    T = 1/n - 10*T; fprintf('%2d %e\n', n, T);  
end  
  
% --- 4. furcsa jelenseg: jegyveszteseg ---  
sqrt(2014) - sqrt(2013)  
1 / (sqrt(2014) + sqrt(2013))  
  
% --- asszociativitas serulese ---  
(1e16 + 1) - 1e16    % 0  
1e16 + (1 - 1e16)    % 1
```

Mikor melyik típust válasszuk?

Feladat	Ajánlott ábrázolás
általános tudományos/mérnöki számítás	double (binary64)
grafika, játék, jelfeldolgozás	float (binary32)
neurális hálók tanítása/inferencia	binary16, bfloat16
pénzügyi elszámolás	egész (fillér) vagy decimal
hosszú, rosszul kondicionált számítás	binary128, tetsz. pontosság
garantált korlátok kellenek	intervallum-aritmetika

- **Túl kicsi** pontosság $\Rightarrow$ hibás eredmény; **feleslegesen nagy** $\Rightarrow$ pazarló memória- és időigény.
- Alternatív irányok: decimális lebegőpontos aritmetika (DFP), intervallum-aritmetika, tetszőleges pontosságú könyvtárak (MPFR, mpmath), reprodukálható összegzés.

- A gépi számok halmaza **véges** és **egyenetlen**: $\mathbb{M}(t, k^-, k^+)$, nevezetes értékei $\varepsilon_0, \varepsilon_1, M_\infty$.
- Az fl input függvény relatív hibája korlátos: $\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{\varepsilon_1}{2}$ — ez a **legjobb, amit remélhetünk**.
- Az IEEE 754 egységesítette az ábrázolást, a kerekítést és a különleges értékeket ($\pm 0, \pm \infty, \text{NaN}$, szubnormálisok).
- A hibák terjedésének szabályai megjósolják, hol lesz baj: szorzás/osztás $\rightarrow$ relatív hibák összeadódnak; kivonás közeli számokból $\rightarrow$ **jegyvesztés**.
- Különítsük el a **kondicionáltságot** (feladat) és a **stabilitást** (algoritmus).
- A gyakorlati receptek: átalakítás jegyvesztés ellen, összegzési sorrend, kompenzált összegzés, tűréshatáros összehasonlítás, megfelelő típusválasztás.

- [1] Lócsi Levente: *Numerikus módszerek 1.* — 1. előadás: Gépi számábrázolás, hibaszámítás. ELTE IK.
- [2] David Goldberg: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys, 23(1), 1991.
- [3] *IEEE Standard for Floating-Point Arithmetic*, IEEE 754-2019.
- [4] Nicholas J. Higham: *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.
- [5] Muller et al.: *Handbook of Floating-Point Arithmetic*, Birkhäuser, 2018.
- [6] ITszotar.hu: *Lebegőpontos számábrázolás (Floating Point): a módszer magyarázata és jelentősége*, 2025.